



Digital Signal Processing Lab

Digital Signal Processing

DP Lab

May 21, 2026

Hochschule für Angewandte Wissenschaften Hamburg
Hamburg University of Applied Sciences

© 2026 Copyright

ANDREA KUPKE, PROF. DR.-ING. ULRICH SAUVAGERD, PROF. DR.-ING. LUTZ LEUTELT
Hochschule für Angewandte Wissenschaften Hamburg,

All rights reserved.

Alle Rechte, auch das des auszugsweisen Nachdrucks, der auszugsweisen oder vollständigen Wiedergabe, der Speicherung in Datenverarbeitungsanlagen und der Übersetzung, vorbehalten.

Dieses Dokument wurde mit Hilfe von KOMA-Script und L^AT_EX gesetzt.

Contents

1	Sampling and quantization	5
1.1	Objectives of this first lab session	5
1.2	Lab preparation	5
1.2.1	Interrupt handler and bit manipulation	6
1.2.2	Sampling and quantization	6
1.3	A first DSP project with Code Composer Studio	7
1.3.1	Start of CCS and import of a project	7
1.3.2	First test of the project	7
1.3.3	Overflows	8
1.3.4	Quantization	8
2	Radix-2 FFT and Real-Time Spectrum Analyser	11
2.1	Objectives of this second lab session	11
2.2	Preparation of the lab	11
2.2.1	Analysis of a Butterfly	12
2.2.2	8-point FFT (DIT)	12
2.2.3	Familiarize yourself with the lab project	14
2.3	Lab: Spectrum Analysis using FFT	15
2.3.1	Getting started with the c project	15
2.3.2	Extension of the FFT to 64 points	15
2.3.3	Real-time spectrum analyser	16
3	FIR Filter Implementation in MATLAB and in C	19
3.1	Objectives of this third lab session	19
3.2	Preparation	19
3.2.1	Sampling frequency and aliasing	20
3.3	Filter Design and Implementation using MATLAB	20
3.3.1	How to create header files in MATLAB for short int coefficients	20
3.3.2	MATLAB design and simulations of lowpass and bandstop filters	21
3.4	Real-time Fixed-Point Implementation and measurements	23
3.4.1	Examination of the hardware: Amplitude response UniDAQ2 board	23
3.4.2	Simultaneous FIR filter implementation of lowpass and highpass filter in ANSI C	23
3.4.3	Bandstop and bandpass filters	24
3.4.4	Code profiling	24



Sampling and quantization

1.1 Objectives of this first lab session

The purpose of this first lab project is to give an introduction to the hardware and software of the UniDAQ2 Digital Signal Processor board, which is used in this and all subsequent lab sessions.

The document *Getting Started* [1] serves as a basis and reference.

You will step by step

- import a Code Composer Studio (CCS) project for the UniDAQ2 board,
- compile and link the project and execute your project on the DSP Client,
- use the CCS debugging tool and correct errors in the source code,
- use interrupt service routines,
- get to know the Interface to ADC and DAC and the usage of hardware interrupts
- and develop simple DSP programs which read audio signals from an audio source and output them through a DAC (directly or after processing).

1.2 Lab preparation

It is very important that you work through these lab instructions before the lab session and that you are familiar with the fundamentals of “Signals and Systems 1+2” and “Programming in C”. If you need to catch up, please make yourself familiar with these topics of the previous semesters.

- In particular, answer all the preparation tasks in the light blue boxes (“Prep task”).
- Familiarize yourself with the document *Getting Started* [1] so that when you get to the lab, you will know for sure what information to look up in it.

Prep task (for lab entry test)

Familiarize yourself with the concepts of the chapter “DP01: Digitization and Digital Signals”, particularly

- sampling, sampling frequency, aliasing and quantization,
- DSP system UniDAQ2 board, interrupt-based sample-by-sample processing in C
- rounding of fixed-point numbers and techniques in C to avoid overflows after arithmetic operations

These topics will be addressed by the lab entry test at the beginning of the lab session.

1.2.1 Interrupt handler and bit manipulation

In your microcontroller class, you have learned how to do bit manipulation of integer values with bit masks and bitwise-logic operators (e.g. and, or, xor). Let an interrupt handler, which is called with every new pair of samples, perform a bit manipulation.

Listing 1.1: bit-mask_unidaq.c.

```
1 interrupt void adcInt (void)
{
3     sData[0] = PRU_addaRegs->adc[0]; // read from ADC channel 0
    sData[1] = PRU_addaRegs->adc[1]; // read from ADC channel 1
5
    sData[0] &= 0x5555;
    sData[1] &= 0xCCCC;
7 }
9 interrupt void dacInt (void)
{
11     PRU_addaRegs->dac[0] = sData[0]; // write to DAC channel 0
    PRU_addaRegs->dac[1] = sData[1]; // write to DAC channel 1
13 }
```

Prep task 1: Interrupt handler and bit manipulation

- Which decimal(!) values are output after bit manipulation to channel 0 and channel 1 of the DAC, if the hexadecimal values received from ADC in the format `int16_t` were `0xFC7F` at channel 0 and `0x83EE` at channel 1?

1.2.2 Sampling and quantization

Let an analog cosine signal $x(t) = \cos(2\pi f_0 t)$ with $f_0 = 4\text{kHz}$ be sampled at $f_S = 32\text{kHz}$. (In the lab you later use a different sampling frequency.) The sampled discrete-time signal $x[n]$ is afterwards quantized by a 4-bit quantizer with amplitude input range $R_{ADC} = [-1, +1]$.

Prep task 2: Sampling and quantization

- Determine the sampled discrete-time signal $x[n]$ (without quantization).
- Determine the eight signal values $x[n], \hat{x}[n]$, $n = 0, \dots, +7$ **before and after** 4-bit quantization with truncation.

1.3 A first DSP project with Code Composer Studio

1.3.1 Start of CCS and import of a project

- Start up the UniDAQ2 board according to the instructions in *Getting Started* [1] and run the prepared program that reads values and outputs them unchanged.
- Set the sampling rate of the board to $F_s = 50$ kHz.

1.3.2 First test of the project

The demo program `main_adda_simple_Lab.c` copies the data of the two ADC registers in the ADC interrupt service routine (ISR) `adcInt` to `sData[0]` and `sData[1]`. These data are now available for processing. In the DAC ISR `dacInt`, the values from `sData[0]` and `sData[1]` are written to two DAC registers.

Lab task 1.1: Feeding the ADC input directly to the DAC output

In this first task, you apply a signal to the ADC and use the given program to read this signal into the DSP and output the signal at the DAC.

1. Function test of the program

- Use the HAMEG HMF2525 function generator to apply a sinusoidal voltage to the input of the board. Mind that you have to **terminate the coax cable from the function generator with a 50 Ω resistor** as otherwise the double value of the set voltage is applied to the DSP board and overvoltages might electrically damage the ADC input.
- Feed a sine wave from the function generator to the ADC 1 input of the board with $V_{pp} = 1$ V and connect an oscilloscope to both output channels. The output DAC 1 should be almost equal to the input signal, at DAC you will see no output.
- Now reconnect the cable from the generator so that the signal is fed to ADC 0. Check whether you are now measuring the sine wave at DAC 0.
- Display the input and output signals at ADC 0 and DAC 0 on the oscilloscope, determine the delay between both sine signals and document the measured delay value and a screenshot of the oscilloscope measurement in the report.

2. Masking

- Mask out channel 0 (set all 16 bits to 0) by inserting the following line between reading and writing the data:
`sData[0] &= 0x0000;`
 - Call up *Run→Debug* to test the program: Channel 0 should now be "silent".
 - Comment out the mask after this exercise.
3. Copy data of a channel
- Now insert the following line before writing the data: `sData[0] = sData[1];`
 - The data from channel 1 is now copied to channel 0 and written to the DAC. Call *Run→Debug* and check the function in a suitable way here too.
 - Comment this line out again.
4. Swap channels
- Ensure that the audio channels are output in reverse: the sine wave fed into ADC 0 should appear at the DAC 1 output. If you feed in at ADC 1, you will only see a signal at DAC 0.
 - **The swapping of the channels must be demonstrated to the supervisors in the lab. Give the code of interrupt handler `dacInt()` including your modifications in the report.**

1.3.3 Overflows

We now want to generate an internal number range overflow by multiplying the values of ADC input 0 by an increasing factor. Use the function generator to apply a sine wave of 300 Hz, $V_{pp} = 1\text{ V}$ to ADC input 0.

Lab task 2: Number range overflows

- Modify the DAC interrupt handler `dacInt()` that the values of both ADC inputs are multiplied by a factor `scale` (defined as a global variable) before they are output to the DAC outputs.
- Add the factor `scale` to the *Expressions window* of the CCS Debugger.
- Increase the factor `scale` in the *Expressions window* until you observe an overflow on the oscilloscope. **Make an oscilloscope screenshot right before and right after the overflow occurs. Specify the value of `scale` at which the overflow occurs and explain the signal shape in the event of an overflow in the report.**

1.3.4 Quantization

We now want to give speech signals into the system and examine the speech quality at different bit resolutions. To do this, both channels are masked with bit masks as in the prep task before they are output to DAC outputs 0 and 1.

Connections to the DSP board. The output of the PC's sound card must be connected to the input of the DSP board via an adapter cable (3,5mm male audio jack to 2 x BNC).

The speakers are connected to DAC outputs 0 and 1 via adapter cables, too (2 x BNC to female audio jack). For simultaneously displaying on the oscilloscope, you must use T shaped BNC splitters at the oscilloscope inputs.

Audio files. Audio files can be found in directory D:\wavefiles\. Use for this task THEFORCE.wav as signal input. Play it back with the PC application *Audacity*.

Lab task 3: Quantization of speech signals

- Make sure that the audio signal is well leveled by leaving the value of factor *scale* as you determined it in task 2, now applied to both channels. Now increase the volume on the PC as much as possible without overflowing (you would hear this in the signal).
- Add a global variable *bitmask* to your program that manipulates both channels

```
sData[0] &= bitmask;  
sData[1] &= bitmask;
```

after your program has scaled both ADC input signals with factor *scale*.

- Add variable *bitmask* to the CCS *Expressions window* and chose a hexadecimal representation by right-clicking on the variable type.
- Give the bit masks required for 1-, 4- and 8-bit quantization as hexadecimal values in the report. Hint: the least significant bits of both channels must be masked out. Is the quantization done by truncation or by arithmetic rounding?
- Set the bit masks in the *Expression window* to the corresponding values for 1, 4 and 8-bit quantization and **compare the intelligibility in the report. Take an oscilloscope screenshot of one 4-bit quantized signal for the report.**



Radix-2 FFT and Real-Time Spectrum Analyser

2.1 Objectives of this second lab session

In this lab, you will implement a 64-point Radix-2 FFT on the signal processor based on a given 8-point FFT. Eventually, you will develop a real-time spectrum analyzer using this FFT implementation. After this lab you should

- better understand the Radix-2 FFT algorithm,
- be able to understand how to implement and execute an FFT on a DSP under real-time constraints,
- be able to implement a framework around an existing FFT algorithms in assembly language in order to perform a frequency analysis of a signal.
- be able to apply a Hamming window to a block of N samples stored in a corresponding buffer

2.2 Preparation of the lab

Prepare well the fundamentals presented in the lecture on DFT and FFT and the preparation tasks in this lab assignment.

Prep task (for short test)

Familiarize yourself with the concepts of

- Discrete Fourier Transform (DFT) and Fast Fourier Transform (FFT), including
- DFT theorems,
- DFT symmetries, and

- effects of windowing.

These topics will be addressed by the short test at the beginning of the lab session.

2.2.1 Analysis of a Butterfly

In Prep Task 1, we analyze the butterfly of the 2-point FFT which is depicted in Figure 2.1.

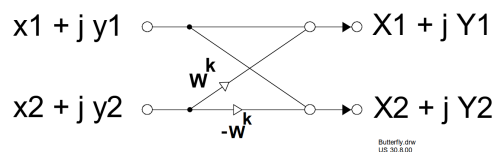


Figure 2.1: Butterfly

Prep task 1

- The relation between the (generally complex) time-domain values

$$z_1 = x_1 + jy_1 \quad \text{and} \quad z_2 = x_2 + jy_2$$

on the left side of Figure 2.1 and the corresponding values

$$Z_1 = X_1 + jY_1 \quad \text{and} \quad Z_2 = X_2 + jY_2$$

of the DFT spectrum on the right side shall be found. Before doing so, please mind:

- Four equations are wanted: two for the real-parts X_1, X_2 and two for the imaginary-parts Y_1, Y_2 .
- The twiddle factor is given by $w^k = e^{-j2\pi k/N}$ and the DFT length is $N = 2$. What is the value of k needed here? Determine the value(s) of the twiddle factor(s).
- Give now the four equations for X_1, Y_1, X_2, Y_2 .
- Rewrite the equations for X_2, Y_2 using only x_1, X_1, y_1, Y_1

2.2.2 8-point FFT (DIT)

An 8-point FFT (DIT) is illustrated in Figure 2.2. Analyse this signal-flow diagram by solving the prep tasks.

The input sequences $x_1[n]$, $x_2[n]$ (not $x_{in}[n]$!!) consist each of the following **8 real decimal** values, which we assume to be stored as 16 Bit (short int):

$$x_1[n] = \{2000, 0, -2000, 0, 2000, 0, -2000, 0\}, N = 0, \dots, 7$$

$$x_2[n] = \{10000, 0, -10000, 0, 10000, 0, -10000, 0\}, N = 0, \dots, 7$$

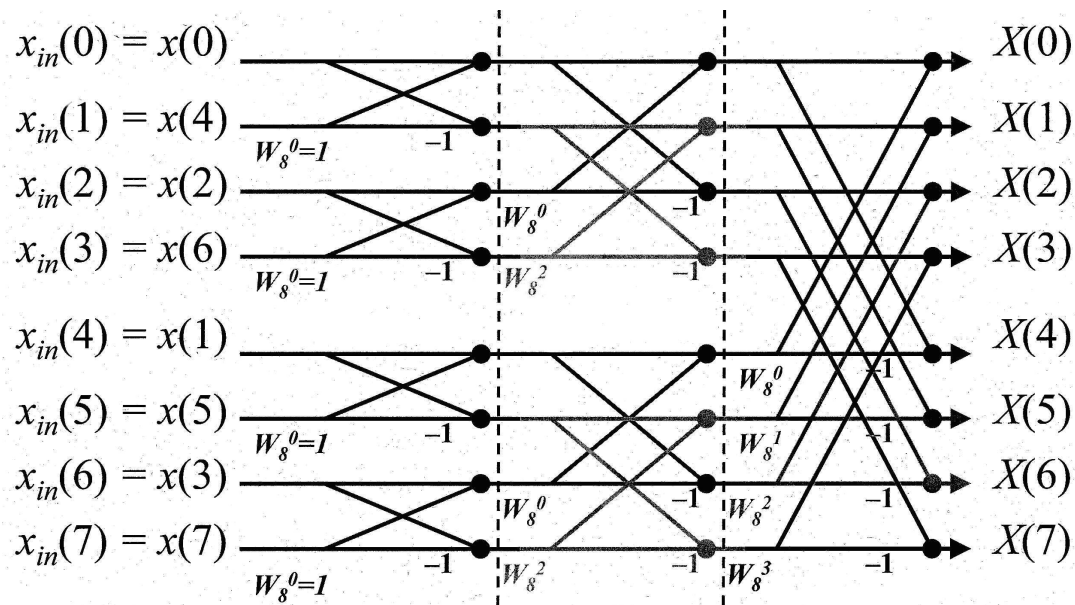


Figure 2.2: 8-point FFT (3 stages)

Prep task 2

1. Put the values of $x_1[n]$ in the correct order according to Figure 2.2. Calculate (e.g. by hand) the output values of the first, second and last stage according to Figure 2.2 and assign the values to the nodes in the graph.
2. Write a MATLAB script `FFT_a.m` which calculates the output signal $X_8[k], k = 0, \dots, 7$ directly (i.e. internal node values not required) using MATLAB's FFT function. Compare your results from above with the result of MATLAB.
3. Do overflows occur?
4. Now repeat the handwritten calculation of the output values of all three stages for $x_2[n]$.
5. Extend your script `FFT_a.m` to calculate the FFT of $x_2[n]$ and again compare your calculation with the one from MATLAB.
6. Do overflows occur (values larger than can be represented with signed 16 bit)? If so, explain why!
7. By which factor do we need to scale the input values $x[n]$ that never an overflow can occur at the output of the 8-point FFT when all values are of type `short int`?
8. Find a method that has a smaller loss in precision as the previous one. Hint: consider a scaling of values at nodes *inside* the FFT algorithm. Explain e.g. with an example why the latter method outperforms method where we scale the input values only?

Hint: Begin each MATLAB script with "clear all". This clears the internal Workspace and if necessary resets 'i' and 'j' (previously defined as index variables) back to imaginary numbers, i.e. $i^2 = -1$, $j^2 = -1$.

Complex-valued input signal: Now examine $x_3[n]$, a complex-value test signal (MATLAB notation):

```
x3 = 0.125*cos(2*pi*3*(0:7)/8) + j*0.125*sin(2*pi*3*(0:7)/8);
```

Prep task 3

Extend your MATLAB script as follows:

- Plot the magnitude spectrum $|X[k]|$ of $x_3[n]$. Pay attention to the correct labeling and scaling of the frequency axis k .
- Does the magnitude spectrum show symmetries? Explain your answer.

2.2.3 Familiarize yourself with the lab project

In D:\ti_work or in EMIL you will find the complete C code for calculating an 8-point FFT. To execute this, copy the following three files from directory D:\ti_work\UniDAQ2.DSP-ADDA\Lab_support into the standard project and remove main_adda_simple_Lab.c:

- FFT8_Radix2_ISR.c (main())
- FFT_butterfly.c
- FFT_radix2.c

In main(), the FFT is calculated once before entering the infinite for(;;)-loop. The program provides already an interrupt routine which however just realizes a simple echo program, i. e., the FFT is not executed again.

Please make sure that you understand the program files of the project, particularly...

- how the input signal is generated,
- how twiddle factors are calculated and how they are arranged in bit-reversed order,
- how the FFT function is called including of bit-reversal of the samples in the FFT buffer in main() once.

The files containing the FFT calculation are **FFT_butterfly.c** and **FFT_radix2.c**. The function call in the C code is:

```
// carry out the N-point FFT on array asX[2*N] IN PLACE  
radix2(N_FFT, asX, asWr, asWi);
```

- This algorithm expects the (real and imaginary) samples in $asX[2 * N_{FFT}]$ in bit-reversed order, while the coefficients $asW[N_{FFT}]$ have to be stored in normal order.
- The real part of the twiddle factors is stored on even addresses of the buffer $asW[N_{FFT}]$, the imaginary samples on the odd addresses.

- A block of N_{FFT} samples of the real-valued part of the input signal $asInBuf[]$ is stored bit reversed on even addresses of the FFT buffer $asX[]$. The imaginary parts on the odd addresses are set to zero, since for a real-valued signal the imaginary part is necessarily equal to zero.
- Optional: A Hamming window shall be applied to the samples stored in $asInBuf[]$. A variable $sDoHamming$ shall be used to turn the window on or off.
- After execution of the FFT, the FFT result is stored in the $asX[2 * N_{FFT}]$ buffer. The calculation is done “in-place”, i.e., the same memory is used for FFT input and output data.
- An ANSI C function $int16_t \text{ bitrev}(int16_t \text{ sIn}, int16_t \text{ sNfftStages})$ for bit-reversal is also provided. The second parameter of this function is referring to the number of FFT stages, not to the FFT length.

2.3 Lab: Spectrum Analysis using FFT

2.3.1 Getting started with the c project

The given program correctly calculates the Radix-2 8-point FFT for an input sequence. If necessary, adjust the input values to the already examined input sequence:

$$x_1[n] = \{2000 \ 0 \ -2000 \ 0 \ 2000 \ 0 \ -2000 \ 0\}$$

Import the prepared project into CCS as for the 1st lab session. Copy the three files *FFT8.Radix2.ISR.c*, *FFT_butterfly.c* and *FFT_radix2.c* from *ti_work\UniDAQ2.DSP-ADDA\Lab.support* into the project folder and deactivate *main_adda_simple.Lab.c* via *Exclude from Build*. First check whether the expected results are delivered. This does not need to be documented.

Lab task 1

1. As a second step, enter the input sequence $x_2[n]$ from prep task and check the result. Do overflows occur? Comment on this and explain the values obtained in a brief calculation.
2. Correct the “error” just determined in the program *butterfly.c*, so that overflows are avoided. Check the functionality: Are the output values correct?
3. In *butterfly.c* replace the equations for X2 and Y2 with the equations from the first preparation task. Check that the results remain identical.

2.3.2 Extension of the FFT to 64 points

Your project should now be extended to a 64-point FFT.

First make a copy of the file *FFT8.Radix2.ISR.c* in the project folder and rename it to *FFT64.Radix2.ISR.c*. After that deactivate *FFT8.Radix2.ISR.c* via *Exclude from Build*.

Lab task 2: 64 point FFT

- Test the 64-point FFT with the following signal written directly to *asInBuf[]* and compare the result with that from MATLAB.
 $x4 = 4096 * \sin(2 * \pi * 4 * (0 : 63)/64);$
- Use the graphical display in CCS via *Tools*→*Graph* (instructions see *Getting Started* [1]) to plot the result against a MATLAB plot.

2.3.3 Real-time spectrum analyser

A continuous FFT analysis of N samples of a real signal is to be performed. The input signal is a sine signal coming from a function generator, the output is displayed in the graphical display. The results are displayed on the oscilloscope in the second step. The sampling frequency is **12,5 kHz**.

In the project folder, make a copy of the file *FFT64_Radix2_ISR.c* and rename it to *FFT64_Analyser.c*. Then disable *FFT64_Radix2_ISR.c* via *Exclude from Build*.

The algorithm is to be implemented as follows:

1. Reading samples

Reading the samples has to be implemented in the ISR.

- The samples from the ADC are stored in a *int16_t* input buffer *asInBuf[N]*. The 0th sample value is saved in *asInBuf[0]*, the 1st in *asInBuf[1]* and so on. During N interrupts, the input buffer is therefore gradually filled with N samples read in.
- A global counter variable *sSamplecount* holds the number of samples already read from the A/D converter.
- If (*sSamplecount* $\geq$ N),
 - a) *samplecount* is reset
 - b) the FFT is calculated

This is done in the infinite loop in *main()*, see below.

2. Calculation of the magnitudes of the spectrum

As soon as the input buffer is filled, you calculate the FFT before the next sample value is read. The following steps are carried out for this purpose:

- First each element of the input buffer *asInBuf[N]* is copied (bit reversed) to *asX[2*N]*, but only to those array elements with even numbered indexes. All array elements with odd index (imaginary parts) have to be explicitly set to zero after calculating a 64-point FFT, since after the calculation *asX[2 * N]* is complex!!
- Function *radix2()* is called and computes the FFT of the last N read samples, stored in *asX[2 * N]*.
Before calculating the FFT, *asX[2 * N]* contains the values for the FFT (*int16_t*); after the FFT, it contains the (complex) values of the spectrum.
- After that, the magnitudes of the spectrum are calculated from *asX[2 * N]* and saved in the output buffer *alOutBuf[N]*. *alOutBuf[N]* now contains the 32 Bit *int* results

of the last read samples as squares of the absolute values.

■ Please note:

- a) Do **not use any printf calls** in interrupt mode.
- b) The twiddle factors are only calculated once, as they do not change.

3. Visualization of the results

The visualization is shown in the graphical display.

Hint: To save time of taking the square roots in the calculation of the magnitudes, it is sufficient to send the *squares* of the magnitudes of the spectrum, i.e. $|X_k|^2$ instead of X_k to the DAC.

■ For the visualization, *Refresh On Halt* and *Enable Continuous Refresh* must be activated in the Graphical Display.

4. Output of the results to the oscilloscope

The output of the magnitude squares and the trigger pulse to the DAC is, of course, also carried out in the ISR.

- During each cycle, the interrupt routine sends one sample from *asOutBuf[]* to channel 0 of the D/A converter. So while reading N new samples, the result consisting of N squared magnitudes of the computed FFT is sent to the DAC.
- Trigger for the presentation on the scope:
Furthermore, if (*samplecount* <= 2), a trigger impulse 32767 is sent to channel 1 of the DAC; otherwise the output is '0'.

Lab task 3: Real-time spectrum analyser

Implement the analyzer according to the description of the algorithm above.

Verify that the FFT64_Analyser.c functions correctly:

Connect the signal generator to the DSK board and select "Waveform Sinus". Choose an amplitude of $2 V_{pp}$.

1. Use the CCS 'graphical display' to monitor the results of the FFT. Start the program, updating the "graph display" as described above. The display should adjust when you change the frequency of the generator.
Take a screenshot for $f_{in} = 1 \text{ kHz}$.
2. Now change the input frequency to $f_{in} = 15 \text{ kHz}$. Save a screenshot and explain in one sentence what you see.
3. In a next step, display the result on the oscilloscope (connect DAC channels 0 and 1 to the oscilloscope and use channel 1 of the board as trigger source). Take screenshots of the scope for $f_{in} = 0.5 \text{ kHz}$ and $f_{in} = 2 \text{ kHz}$
4. *Optional:* Compute in MATLAB a 64-point Hamming-window and scale it to a *int16_t* variable *asHammWind*[64]. Multiply *asInBuf*[] with this window before the buffer *asInBuf*[N] is copied to *asX*[2 * N]. Create a variable *sDoHamming* to switch the windowing on and off.
Connect a sine signal of amplitude of $2 V_{pp}$ and frequency 500 Hz to the input of the DSK board. Display the output buffer in the CCS „graph display“. Set a breakpoint at

the line where samplecount is set to zero.

Start the program, updating the “graph display” at the breakpoint. Display the variable *sDoHamming* in the CCS “Expressions Window” and switch *sDoHamming* on and off. Comment on the effect of the Hamming-window on the FFT output in *alOutBuf[]* (magnitude spectrum displayed logarithmically in a CCS “graph display”)



FIR Filter Implementation in MATLAB and in C

3.1 Objectives of this third lab session

The purpose of this lab is to design and implement in real-time FIR digital filters (FIR: finite impulse response) on the UniDAQ2 board. MATLAB will be used to design and evaluate FIR filters, and to simulate them in time domain. After this lab you will:

- be familiar with FIR filter design methods,
- be able to design an FIR filter in MATLAB,
- be able to implement an FIR filter in fixed point arithmetic on a DSP.

3.2 Preparation

Prepare well the fundamentals presented in the lecture on FIR filters and the preparation tasks in this lab assignment.

Prep task (for short test)

Familiarize yourself with the concepts of

- Finite Impulse Response (FIR) filters, including
- description of FIR filters by impulse response, transfer function, frequency response, pole zero plot and
- properties like symmetries, linear-phase and group delay.

These topics will be addressed by the short test at the beginning of the lab session.

3.2.1 Sampling frequency and aliasing

Assume the real-time DSP system consisting of Audio Codec and DSP has a non-ideal low-pass characteristic as depicted in Figure 3.1. An analog cosine signal $x(t) = 1/\pi * \cos(2\pi f_0 t)$ is applied to the system input at different frequencies f_0 . Let the sampling frequency f_S of the Audio Codec be 24 kHz.

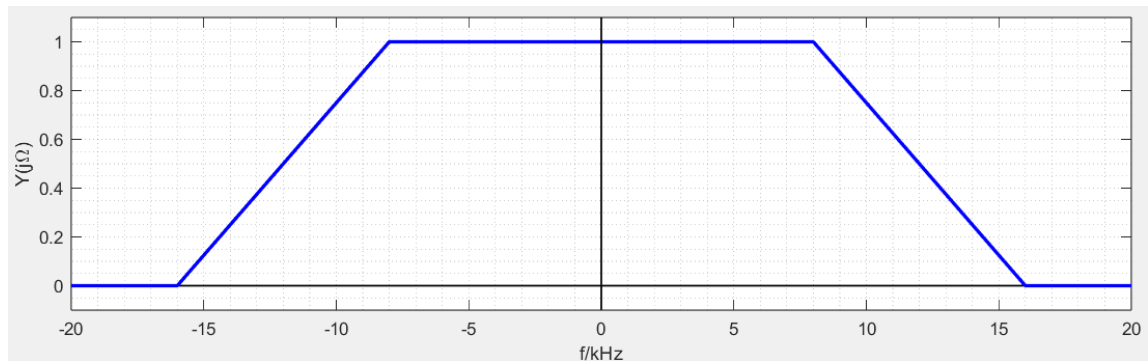


Figure 3.1: Low-pass characteristic of a DSP system

Prep task 1: Sampling frequency and aliasing

- Explain for what purpose a DSP system has a low-pass filter characteristic.
- Sketch the spectrum $Y(j\Omega)$ of the analog signal $y(t)$ at the output of the DSP system with different colors for $f_0 = 6$ kHz, 10 kHz, 12 kHz and 18 kHz (in the same plot). It is *not* necessary to calculate the exact spectral amplitudes.

3.3 Filter Design and Implementation using MATLAB

3.3.1 How to create header files in MATLAB for short int coefficients

The signal flow diagram of an FIR filter is shown in Figure 3.2 for a filter degree of $N = 3$, thus 4 coefficients.

The listing 3.1 shows how to write a header file “FIR_LP.h” for the filter coefficients in MATLAB. MATLAB coefficients for FIR filters are usually in the range between -1.0 and +1.0, where +1.0 should be excluded because +32768 does not exist as *int16_t*.

- Function *write_coeff* stores the FIR filter coefficients in an array *int16_t b_FIR_coe*.
- The header file thus contains *int16_t* coefficients in the range -32768 ... +32767.

Listing 3.1: Store MATLAB FIR filter coefficients in a header file

```
1 % How to create header file FIR_LP.h
2 % Assumption :
  % The coefficients of an FIR filter with N_Coe filter coefficients are
4 % stored in b_FIR_coe. The FIR filter thus has a degree of (N_Coe-1)

6 function write_coeff(fid, asFirCoeLP, b_fir_coe, N_Coe);
```

```

    fprintf(fid, 'int16_t');
8  fprintf(fid, asFirCoeLP);
    fprintf(fid,['[',num2str(N_Coe),']']={'\n'});
10
    k = 0;
12 for p= 1: N_Coe;
        fprintf(fid,'%6.0f,', round(b_fir_coe(p)*32768) );
14     k = k + 1;
        if k >5
16         fprintf(fid, '\n');
            k = 0;
18     end
    end
20 fprintf(fid, '};\n\n');

22 %-----

24 % How to call the function for example for low pass filter:

26 fid = fopen('FIR_LP.h', 'w');          % open the header file
    fprintf(fid,'#define N_FIR_LP_COE %d\n', N_Coe);
28 write_coeff(fid, 'asFirCoeLP', b_FIR_coe, length(b_FIR_coe) );
    fclose(fid);                          % close the header file

```

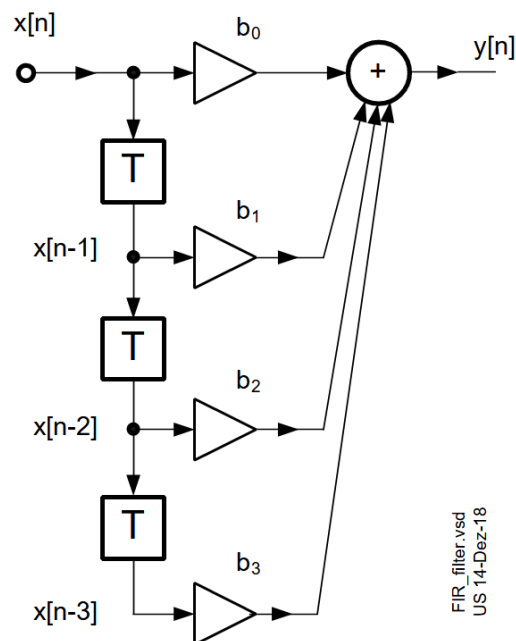


Figure 3.2: Signal flow diagram of an FIR filter

3.3.2 MATLAB design and simulations of lowpass and bandstop filters

First, an FIR lowpass filter is to be designed. From this the coefficients for a bandstop filter are to be derived.

Note: In the following, make sure that the lowpass filter has a odd number of coefficients, thus filter degree is even.

To adjust the filter degree accordingly, you can change it after executing *firpmord*() before passing it to *firpm*().

Specification of the lowpass filter to be designed:

passband edge frequency	4500 Hz
stopband edge frequency	5000 Hz
maximum passband ripple	0.1 (this is NOT in dB)
minimum stopband attenuation	40 dB
sampling frequency	25 kHz

Design of a bandstop filter using frequency transformation::

Sub-sampling turns a low-pass filter into a bandstop filter: One replaces " $H(z)$ " by " $H(z^2)$ " using the frequency transformation.

Prep task 2: lowpass/bandstop filter design

1. Write a MATLAB script *FIR_lab.m* that designs a lowpass filter with the above specification. Use the MATLAB functions *firpmord*() and *firpm*().
2. Determine the coefficients of an bandstop filter from the designed lowpass filter and report the coefficients of both filters.
3. Plot the impulse responses of the lowpass filter *b_fir_coe_LP* and the bandstop filter *b_fir_coe_BS*.
4. Specify the four edge frequencies of the bandstop filter. These result from the edge frequencies of the lowpass filter.
5. Plot the amplitude responses in dB with coefficients corresponding to the precision of *int16_t* (16 bits). Use "round" in combination with an appropriate scaling factor.
6. Check all parameters of the specification. If it is not complied with, the filter order may have to be increased. This would have to be done after calling *firpmord*().
7. Create *int16_t* (16 bits) filter coefficients and write each to a header file, *FIR_LP.h* resp. *FIR_BS.h*. Use the function *write_coeffs.m* for this, which is provided in Moodle. Use the code lines for calling the function at the end of Listing 3.1.
8. Quantize the coefficients of the lowpass filter to 8 bit precision. Plot the amplitude responses in dB. What attenuation do you achieve with the quantized coefficients in the stopband?

Lowpass to highpass transformation

If a low-pass filter is subtracted from an all-pass filter, a high-pass filter is created. This context is to be applied in the task.

Prep task 3: Highpass

Extend your MATLAB script *FIR_lab.m* to generate the coefficients for a highpass from the lowpass coefficients.

1. Generate the coefficients of an all-pass filter that has the same order as the lowpass filter. Use the example discussed in the lecture.
2. Subtract the lowpass coefficients from the allpass coefficients.
3. Plot the amplitude response of the resulting highpass filter in dB.
4. Give the full specification of the high-pass filter and explain how this is achieved.
5. Create *short int* filter coefficients and write them in a header file *FIR_HP.h*.

3.4 Real-time Fixed-Point Implementation and measurements

3.4.1 Examination of the hardware: Amplitude response UniDAQ2 board

Hint: For the following measurements you first need the introduction to the audio analyzer UPV. Start even with point 1 of the task.

If you need to wait, you can start implementing the filter.

Lab task 1: Amplitude response

1. Use again the starter project of the 1st lab and rename the file *main_adda_simple_Lab.c* to *fir_Lab.c*.
2. Measure the amplitude response of the board.
Briefly explain what you observe.

3.4.2 Simultaneous FIR filter implementation of lowpass and highpass filter in ANSI C

In this task, you extend the ISR of a c program in CCS to calculate the FIR low-pass and high-pass filters.

The constraints for the implementation are as follows:

- Both filters are to be applied to the same input signal *int16_t sX* and also use the same delays *asFirDelays[]* in which the previously read input samples are stored.
- The implementation is done in fixed-point arithmetic.
- If you use local variables in the ISR, they must be declared as *static*.

Lab task 2: realtime implementation

1. Integrate the header files containing the FIR lowpass and highpass filter coefficients by using `#include "fir_LP.h"` and `#include "fir_HP.h"`. These two files have to be stored in the project directory.

2. Use ADC channel 0 as input and implement the high pass filter first, use DAC channel 0 as output.
3. Verify the functionality of the filter with the UPV.
4. Then add the low pass filter to the implementation. Use DAC channel 1 as output for this. Display both amplitude responses on the UPV, use both measurement channels for this.
5. Comment on the results and explain in particular the differences in stopband attenuation and passband ripple.

3.4.3 Bandstop and bandpass filters

The bandstop is implemented in this task. To turn the bandstop into a bandpass, every second coefficient must be inverted - or you can subtract every second product during the calculation instead of adding it.

Lab task 3: Bandstop and bandpass

1. Implement the bandstop and display the amplitude response on the UPV.
2. Extend the program for a bandpass: In a branch, the calculated product must now be subtracted for every second coefficient. Ensure that each coefficient is only multiplied once.
3. Display the bandstop and bandpass together on the UPV.
4. Name the cutoff frequencies of the bandpass filter. Why do they not correspond to those of the bandstop filter?

3.4.4 Code profiling

Profiling can be used, among other things, to determine exactly how many clock cycles programmed code requires. This is useful to verify whether written code can be executed within an ISR. This will be checked here.

Cycles needed for interrupt handling can be ignored for the following questions.

Lab task 4: Code profiling

- First calculate how many clock cycles are available in one sampling period. (The TMS320C6747 is clocked with 456 MHz.)
- Measure the number of clock cycles needed to compute two filters by profiling. A short instruction can be found in *Getting Started* [1].
- Determine (only theoretically!) the maximum sampling rate that would be possible to still calculate the two filters correctly.

Bibliography

- [1] Getting_Started_with_Unidaq2_en.pdf: *Introduction and operation of the UNiDAQ2 in the Signal Processing Lab.*
moodle course of the lab
- [2] DSignT: *UniDAQ Processor Board UniDAQ2.DSP-ADDA.*
<https://www.dsignt.de/de/unidaq/unidaq2-dsp-adda.html>
- [3] UPV_Starter_en.pdf: *Introduction Audioanalyser R&S UPV.*
moodle course of the lab
- [4] Datasheet-BM8-May-2021.pdf: *Datasheet Kemo BM 8.*
moodle course of the lab
- [5] S.K.Mitra: *Digital Signal Processing, McGraw-Hill, 2001*
- [6] E.C.Ifeachor, B.W.Jervis: *Digital Signal Processing - A Practical Approach, 2nd ed., Prentice Hall, 2002*
- [7] von Grünigen: *Digitale Signalverarbeitung, Fachbuchverlag Leipzig, 2004*